
$$\begin{array}{c} \backslash \quad \wedge \quad \wedge \\ \backslash \quad (\text{oo}) \backslash \rule{0.8cm}{0.4pt} \\ (\rule{0.6cm}{0.4pt}) \backslash \qquad \qquad) \backslash / \backslash \\ \qquad \qquad | | \text{---} \text{---} w \quad | \\ \qquad \qquad | | \qquad \qquad | | \end{array}$$

Week 14

Exam 2 Prep!

$$\begin{array}{c} \backslash \quad \wedge \quad \wedge \\ \backslash \quad (\infty) \backslash \text{---} \text{---} \\ \quad (\text{---}) \backslash \qquad \qquad) \backslash / \backslash \\ \qquad \qquad | | \text{---} \text{---} w \quad | \\ \qquad \qquad | | \qquad \qquad | | \end{array}$$

FILL THIS OUT!!!!

Taking the Exam 2 Determination Quiz is REQUIRED and DUE BY MIDNIGHT (4/21)!

Due to logistical reasons, not filling out the quiz could lead to us NOT configuring your exam to be visible at all. Not filling out the Exam 2 Determination Quiz is a responsibility on your end and if you forget to fill it out by midnight, we will NOT be accommodating in letting you fill it out later.



Question 1

1 pts

Have you completed the exam 2 determination quiz?

Not sure? Check Here:

Section A: <https://gatech.instructure.com/courses/502376/quizzes/780521>

Section B: <https://gatech.instructure.com/courses/502376/quizzes/780517>

☐ I have completed the exam 2 determination quiz

Topics

Major Pain Points

- Ordering, Atomicity, and Crash Consistency
- Networking
- Concurrency
- Scheduling

Ordering, Atomicity, and Crash Consistency

Why This Matters

Problem: What should the FS guarantee if the machine crashes/powers off/stops unexpectedly?

- Two Core Properties
 - ORDERING: updates happen in the *intended* order
 - ATOMICITY: updates happen *completely* or *not at all*, NEVER partially
- Why Bother?
 - Users want to know when data is actually persisted
 - The FS must avoid corrupting metadata and externally visible state
- Intuition
 - CRASH CONSISTENCY: preserving a sane recoverable state after failure
 - CORRECTNESS: did the write happen & did the right parts become visible in the right order?

Disk Interface

Primitives:

- `write(block)` — may be buffered, may reorder, NOT atomic
- `flush` — waits until all pending writes are synced to disk. *How we enforce ordering*
 - Ordering is achieved only through flush operations. Without flushes, the disk can complete writes in any order
- Ordering Arithmetic (why flushes matter)
 - 4 writes with no flush: $4! = 24$ possible orderings
 - 3 writes + flush + 1 write: $3! \times 1! = 6$ possible orderings
 - General: n writes + flush + k writes: $n! \times k!$ orderings
 - Since $(n + k)! \geq n! \times k!$, partial ordering strictly shrinks the space of possible outcomes
- Crash Risk: If a write is only partially completed when power is cut, the disk head can corrupt the block being written. You may not get old data OR new data, you get garbage.

Atomicity via **Shadowing**

CORE IDEA: Make the “commit” a single-bit decision between two *complete* copies of data

HOW IT WORKS:

1. Maintain two copies of data: *current* and *shadow*
2. A single shadow bit indicates which copy is active
3. On update: copy current to shadow, then modify the shadow copy
4. Flush steps *1 and 2* to disk
5. Flip the shadow bit (this is the atomic commit)

Why is this atomic?

- The shadow bit is a single Boolean write. It either flips or it doesn't (NO PARTIAL STATE)

Atomicity via **Shadowing**

Crash Analysis:

1. Crash before bit flip (before step 5)
 1. Shadow copy has new data
 2. Current copy has old data
 3. Bit still points to old
 4. OVERALL: Safe, recover old state
2. Crash after bit flip (after step 5)
 1. New data is now current
 2. OVERALL: Safe, new state is visible

DESIGN COST (IMPORTANT!!!!!!)

- At least 2x storage overhead (very obvious why)
- Every update requires:
 - Write (copy)
 - Write (update)
 - Flush
 - Write (flip bit)
- Shadow bit must live on a *separate* disk block from the data
- Consecutive updates require alternating shadow states →
Added complexity & flushes

Atomicity via **Logging (Journaling)**

CORE IDEA: Keep a sequential log of *intended changes*. Commit atomically via a commit record. On recovery, replay the log.

- Log Structure

[*entry* | COMMIT | *entry* | COMMIT | *entry* (uncommitted)]

→ each *entry* contains: Type | Location | Data

- Rules

- An entry is only valid if it has a COMMIT record after it
- Uncommitted entries are ignored on recovery
- If committed, the log version supersedes what's on the main disk
- Entries are committed in order — if entry *i* is not committed, neither is entry *i* + 1

Atomicity via **Logging (Journaling)** via separate commit block

CORE IDEA: Use a dedicated commit block as the atomic “point of no return.” The flush between the entry write and the commit write enforces a strict happens-before relationship

- How it Works

- | | |
|------------------------|---|
| • write (entry) | : write the log entry (data + location) |
| • flush | : BARRIER: entry <i>must</i> be on disk before commit |
| • write (commit block) | : write the commit record (single block) |

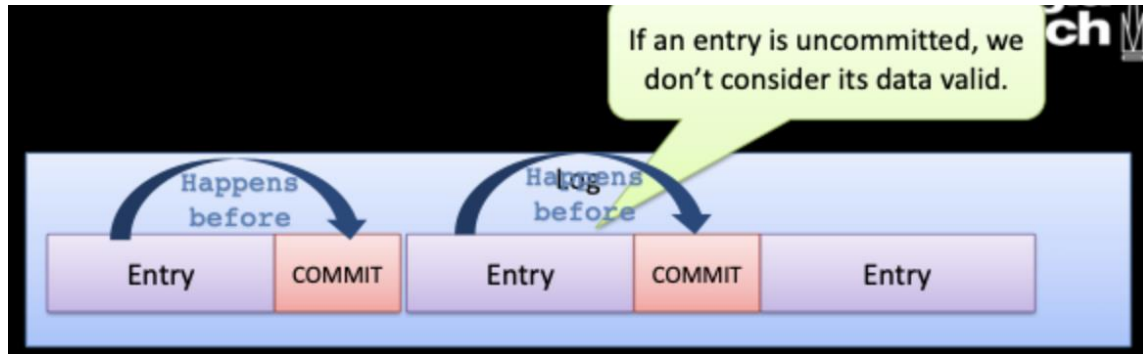
- Why the flush?

- Guarantees the entry is physically on disk before the commit block is written. Without this, the disk could write the commit before the entry and on recovery you’d see a commit with no valid entry.

Atomicity via **Logging (Journaling)** via separate commit block

- What happens on crash at each point

Crash Point	What's on Disk	Recovery Outcome
After Step 1, Before flush	Entry may or may not be on disk	No commit found → entry ignored → safe
After Flush, Before Step 3	Entry is on disk, no commit	No commit found → entry ignored → safe
After Step 3	Entry on disk, commit may or may not be	If commit missing → entry ignored If commit present → entry applied → safe



Each Entry + COMMIT costs 2 writes + 1 flush
- Flushes are expensive! This is a cost of this mechanism

Atomicity via **Logging (Journaling)** via checksum commit

CORE IDEA: Embed a cryptographic checksum of the entry's contents directly into the commit record. A commit is only valid if its checksum matches the entry. This lets multiple entries share a single disk block, eliminating intermediate flushes.

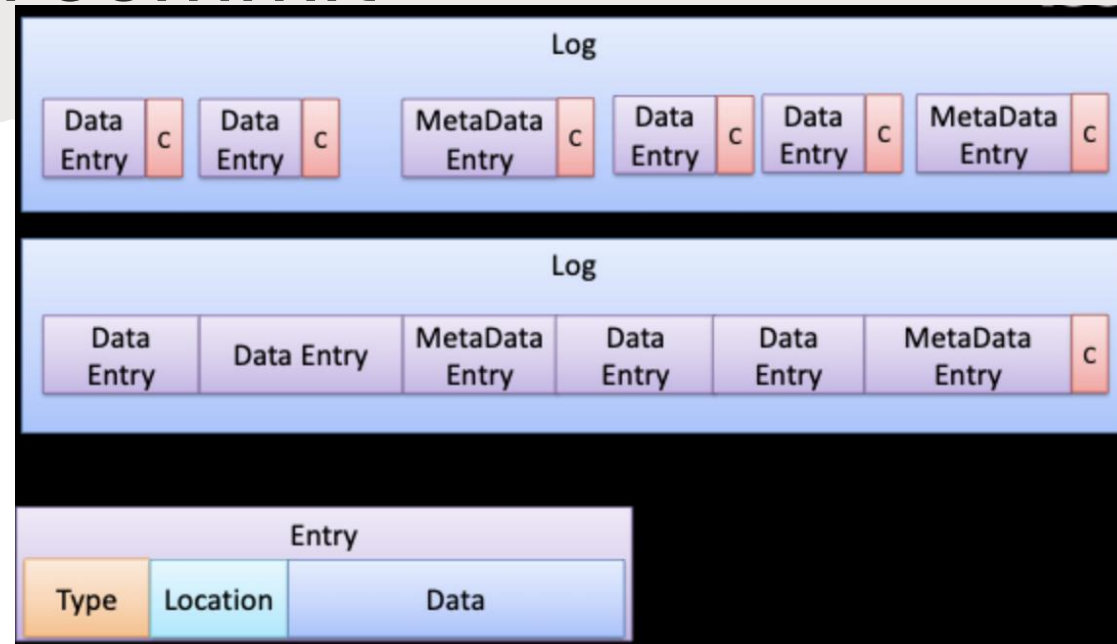
- How it Works

- write (entry + checksum) : single write: data AND checksum together
 - Repeat for as many entries that fit in one block
- There is no flush needed between entries on the same block

Why is no flush needed?

- Because validity is determined by checksum, not by ordering. If the block was partially written (a crash mid-write), the checksum won't match (the entry is automatically treated as uncommitted). The checksum acts as the atomic commit signal, not the presence of a separate block.

Atomicity via **Logging (Journaling)** via checksum commit



- How Partial Write Protection Works:
 - Full Write Scenario: [Entry | Checksum (entry data)] ← checksum matches, COMMIT
 - Partial Write Scenario: [Entry | Checksum (entry data)] ← checksum fails, NOT COMMITTED
- Multiple Entries, One Block, One Write:
 - [Entry A | Checksum (entry A) | Entry B | Checksum (entry B) | Entry C | Checksum (entry C)]
 - one single write() call → all three committed atomically
- Chained Validity Rule: An entry is only committed if ALL entries before it in the log are also committed. So even if Entry C's checksum is valid, if Entry B's checksum fails, Entry C is also treated as uncommitted. This handles out-of-order writes at the block level.

Atomicity via **Logging (Journaling)**

Trade-offs

	Mechanism 1	Mechanism 2
Writes per Entry	2 (entry + commit block)	1 (entry + checksum in one)
Flushes per Entry	1	0
Handles Partial Writes?	No (relies on ordering)	Yes (checksum)
Multiple Entries per Block?	No	Yes
Complexity	Simple	Complex (checksum computations)
Performance	Slower (flush-bound)	Faster (batched entries)

- Checksum both *validates* the write (detecting partial writes) and *commits* the entry (replacing the need for a separate commit block and flush)

Log Properties, Recovery & Log Merge

Properties the Log Gives You:

- Ordering: Entries are appended sequentially; order is implicit
- Atomicity: Commit records make updates all-or-nothing
- Sequential I/O: Logs are written sequentially, which is very efficient on spinning disks
- Fewer flushes than shadowing

Recovery Procedure (on every startup after crash):

1. Scan the log from the beginning
2. Apply all committed entries to the main filesystem data (log merge)
3. Stop at the first uncommitted entry
4. Clear the log
5. FS is now consistent and ready

Log Properties, Recovery & Log Merge

What if we crash during log merge?

- Log merge is itself multiple non-atomic disk writes
- Solution: log entry updates must be idempotent (applying them multiple times produces the same result as applying once)
- On next boot, recovery simply replays the log again (safe because idempotent)

Clearing the log atomically

- The log is only valid up to the first uncommitted entry
- To “clear” the log, just invalidate the first entry
- Recovery will naturally stop there

Shadowing vs Logging Comparison

Property	Shadowing	Logging
Storage Overhead	2x minimum	Log area only
Atomicity Mechanism	Single bit flip	Commit Record
Ordering	Flush before bit flip	Implicit (append-only)
Recovery Complexity	Simple (check shadow bit)	Must Replay Log

fsync

Key POSIX FS calls:

- read/write : basic I/O (writes may be cached, NOT guaranteed on disk)
- fsync(fd) : forces all pending writes for fd to be persisted to disk
 - After fsync: data is on disk
 - Before fsync: no guarantees
- mkdir, create, unlink : directory/file operations

Crash-Safe File Updates

- Why so many fsyncs?

- For ordering. Need to guarantee log is visible on disk before the file update, so recovery can use the log if we crash mid-update

Recovery Logic

- Boot up, check for log file in directory
- If log exists and checksum valid → log write was committed but file write may not have completed → replay log into file
- If no log → either update never started, or completed fully → safe either way

Inefficiency: The first two fsyncs are for ordering. They force disk flushes that may be unnecessary if the disk happens to write things in order anyways

create(log)

write(log, offset, size, data, checksum)

fsync(log)

- Ensure log is on disk before dir entry

fsync("./")

- Ensure dir entry for log file is on disk

write(file, data)

- Actual update

fsync(file)

- Ensure file data is on disk

unlink(log)

- Remove log (update is complete)

fsync("./")

- Ensure dir entry removal is on disk

Optimistic Crash Consistency

- **osync (ordering sync)**
 - Guarantees that writes before the osync are ordered before writes after it, but does NOT force a disk flush.
 - Very cheap (just annotates ordering in the disk queue)
- **dsync (durability sync)**
 - Forces data to physical link
 - Expensive (only called when you need persistence)
- **Why this is faster**
 - The intermediate ordering syncs don't stall the disk (they just tag the I/O operations with ordering constraints). Only the final dsync/fsync forces a true flush
- **Tradeoff**
 - More complex recovery logic but better performance for write-heavy workloads

create(log)

write(log, offset, size, data, checksum)

osync(log)

- Cheap: order log write before dir entry

osync("./")

- Cheap: order dir entry before file write

write(file, data)

osync(file)

- Cheap: order file write before unlink

unlink(log)

fsync("./")

- Only one expensive flush at end

Why fsync is Expensive

When fsync(f2) is called:

- Log must be fully flushed to disk
- Log contains entries for BOTH f1 and f2
- Flushing f2 forces f1's 2GB to flush

Why?

- A shared log places a total ordering on all FS writes. Flushing any part of the log means flushing the whole log up to that point

Consequence

- An fsync on a small file can trigger enormous I/O if other large writes are in-flight. This makes fsync latency highly variable and unpredictable

```
write(f1, 2GB)
```

```
write(f2, 1 byte)
```

```
fsync(f2)
```

Design Questions

1. Head TA Rohan proposes: “We don’t need a separate shadow bit block, we can store the shadow bit in the same block as the data” Why is this incorrect?
2. You have the following log: [Entry A | COMMIT | Entry B | Entry C | COMMIT | Entry D]. The system crashes. On recovery, which entries are applied? What if the COMMIT for Entry B/C had a bad checksum?
3. Design a simple logging scheme for a filesystem that must handle crash-safe renames (`rename("./x/old", "./y/new")`). What entries go in the log? What is the recovery procedure?
4. Explain why log entry updates must be idempotent. Give a concrete example of an operation that IS idempotent and one that is NOT and explain what goes wrong with the non-idempotent one during log replay after a crash mid-merge.

Design Questions

Head TA Rohan proposes: “We don’t need a separate shadow bit block—we can store the shadow bit in the same block as the data” Why is this incorrect?

1. The write is no longer atomic:

1. When you write the block containing both the shadow bit and data, the disk writes the entire block. A crash mid-write could leave the block in a state where the shadow bit has been updated but the data hasn’t. Now you have a block where the shadow bit says to use one copy but the data in that same block is garbage

2. The original problem:

1. Point of the shadow bit was to proxy a large multi-block update with a single-bit flip. If the bit and the data are co-located, you now need the bit flip AND the data write to be atomic together

Design Questions

You have the following log: [Entry A | COMMIT | Entry B | Entry C | COMMIT | Entry D]. The system crashes. On recovery, which entries are applied? What if the COMMIT for Entry B/C had a bad checksum?

All checksums valid:

- Entry A : COMMIT present and valid, apply Entry A
- Entry B : COMMIT present and valid, apply Entry B
- Entry C : same COMMIT, apply Entry C
- Entry D : no COMMIT, do NOT apply Entry D

Bad checksum

- Entry A : COMMIT present and valid, apply Entry A
- Entry B : COMMIT checksum fails, do NOT apply Entry B
- Entry C : same COMMIT, fails, do NOT apply Entry C
- Entry D : no COMMIT, do NOT apply Entry D

Design Questions

Design a simple logging scheme for a filesystem that must handle crash-safe renames (rename("./x/old", "./y/new"). What entries go in the log? What is the recovery procedure?

Atomically needs:

1. Adding a directory entry for new in directory y
2. Removing the directory entry for old from directory x
3. The inode itself doesn't change – just the directory entries

Log Entries needed:

- Entry 1: "Add dirent (name='new', inum=N) to directory y's data block"
- Entry 2: "Remove dirent (name='old', inum=N) from directory x's data block"
- Entry 3: "Update link count of inode N"
- COMMIT

Design Questions

Design a simple logging scheme for a filesystem that must handle crash-safe renames (rename("./x/old", "./y/new"). What entries go in the log? What is the recovery procedure?

Recovery Procedure:

- Find log, validate checksum
- If valid: apply entry1 (add to y), apply entry2 (remove from x), apply entry3 (fix link count) → unlink log
- If invalid: discard log, both directories retain original state (rename never happened)
- Idempotent check: applying "add dirent (new, N) to y" twice is safe (second sees entry already exists, no change). Applying "remove dirent (old, N) from x" twice is safe (second sees entry is already gone, no change)

Design Questions

Explain why log entry updates must be idempotent. Give a concrete example of an operation that IS idempotent and one that is NOT, and explain what goes wrong with the non-idempotent one during log replay after a crash mid-merge.

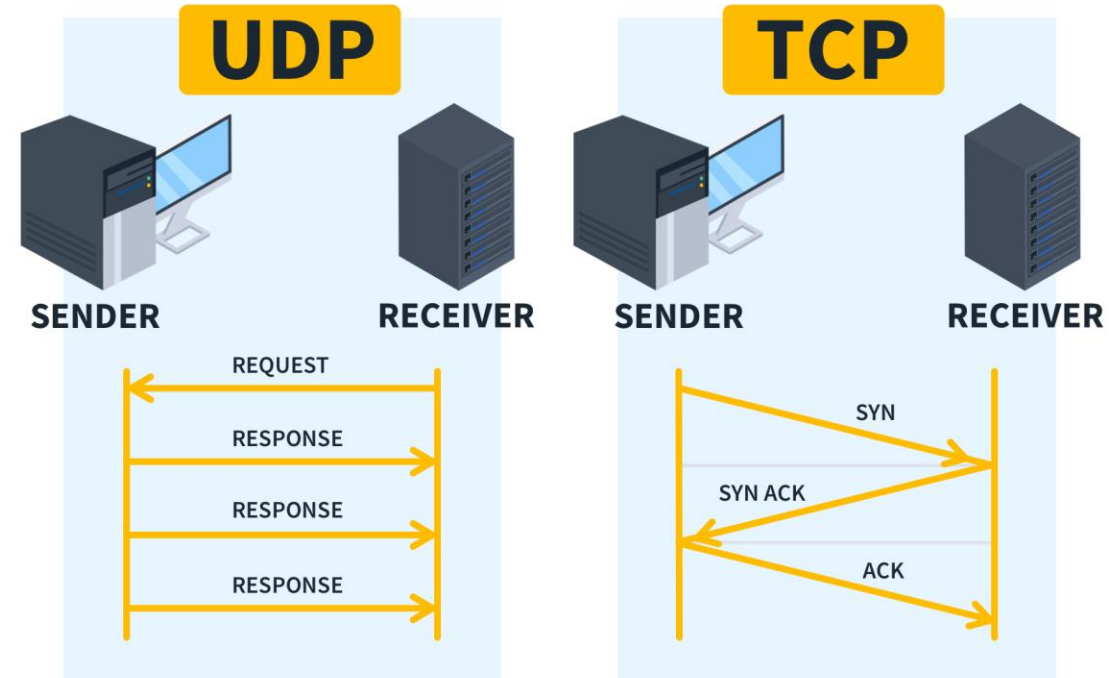
1. If applying an entry twice produces a different result than applying it once, the filesystem ends up in an incorrect state
2. Log Entry: “write the value 42 to block 7, offset 0”
First Apply: block 7, offset 0 → 42
Second Apply: block 7, offset 0 → 42
3. Log Entry: “add 1 to the file size stored in inode 5”
First Apply: inode 5 size: 100 → 101
Second Apply: inode 5 size: 101 → 102

If the system crashes after the first apply but before clearing the log, recovery replays the log and applies the entry again. Now the inode reports a size of 102, but the file only has 101 bytes of data. Reading past offset 101 returns garbage or causes an error.

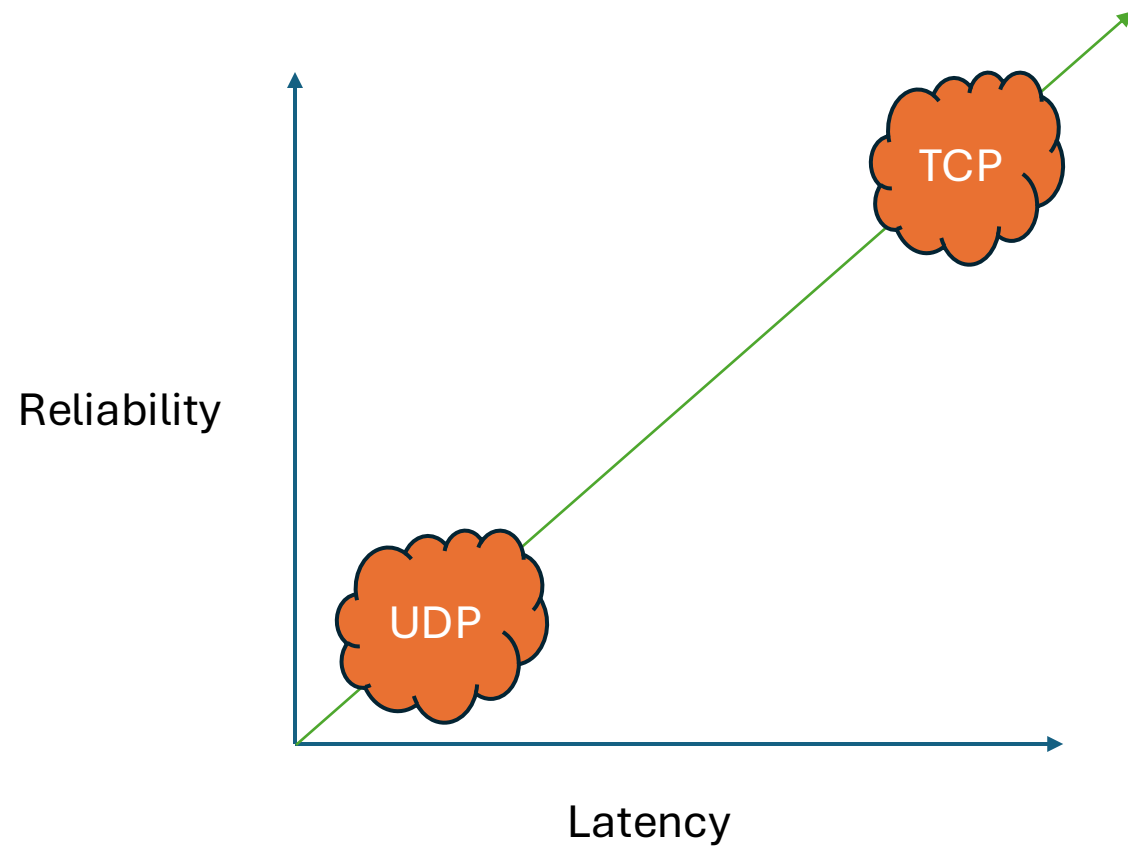
Networking

TCP vs. UDP

- Transport Layer (L4) protocols
- **Transmission Control Protocol (TCP):**
 - Sequence #s
 - Retransmissions/ACKs
 - 3-way TCP Handshake
 - Checksums
 - Stream interface
- **User Datagram Protocol (UDP):**
 - Packets may be reordered or dropped
 - Connectionless
 - Checksums
 - Datagram interface



Tradeoff Space



Scenarios

Goal: Ordering / No Lost Packets

- Text messages
- Databases
- Emails
- Web pages

Scenarios

Goal: Connections

- Bidirectional Communication
 - Text messages!
 - Turn-based video games (e.g. Chess.com, Clash Royale)

Scenarios

Goal: Stream Interface

- Unknown/unbounded length
- Large artifacts
- Examples:
 - File transfer
 - Log aggregation

Design Questions

1. You're designing a stock ticker that broadcasts the latest price to thousands of subscribers every 10ms. Missing an update is fine; stale data is worse than no data. TCP or UDP? Why?
2. You're building a cloud-based file backup service. Files range from 1 KB to 500 GB. TCP or UDP? Why?
3. Video conferencing (e.g. Zoom) sends audio/video over UDP but uses TCP for the signaling channel (e.g. who's in the call, chat messages). Why the split?

Concurrency

Concurrency and parallelism

- Concurrency is about interactions between independently executing threads
 - or, "nondeterministic composition of programs" (Bob Harper)
 - or, "dealing with lots of things at once" (Rob Pike)
- Parallelism is just about doing multiple things at the same time
 - You can have concurrency without parallelism (but not the other way around)
- Where does concurrency come from?
 - On normal computers, two hardware features: interrupts and multiple CPUs (or cores)
 - Plus OS abstractions: processes and threads.
 - **All OS abstractions come from either interrupts or multiple CPUs.** With neither, the system is guaranteed to be single-threaded.

Deadlocks

Recall the conditions necessary for a deadlock to occur:

- **Mutual Exclusion:** Resources must be non-sharable.
- **Hold and Wait:** A thread holds at least one resource while waiting to acquire other resources held by other processes.
- **No Preemption:** Resources cannot be forcibly taken away; they must be released voluntarily.
- **Circular Wait:** A closed chain of threads exists, where each thread waits for a resource held by the next in the chain.

Deadlocks

Mutual Exclusion: Resources must be non-sharable.

Consider a lock that allows $\leq k$ processes to hold it at once.

When would mutual exclusion be satisfied?

Deadlocks

Hold and Wait: A thread holds at least one resource while waiting to acquire other resources held by other processes.

Is there a way you could design locks to mitigate this condition?

Deadlocks

No Preemption: *Resources cannot be forcibly taken away; they must be released voluntarily.*

Self-explanatory.

As an exercise, consider how you could design a program that could safely respond to deadlocks by cancelling the offending threads. (Think back to the filesystems section!)

Deadlocks

Circular Wait: A closed chain of threads exists, where each thread waits for a resource held by the next in the chain.

Is deadlocking possible here?

T1

Lock(a);

Lock(b);

Lock(c);

...

Unlock(c); Unlock(b); Unlock(a);

T2

Lock(c);

Lock(a);

Lock(b);

...

Unlock(c); Unlock(b); Unlock(a);

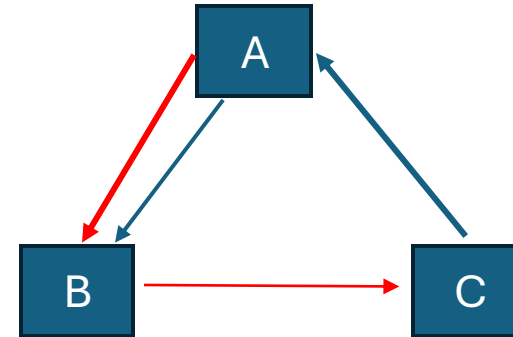
Yes; let's try to see this visually.

Deadlocks

Make a graph with one node per lock.

Add edges in the order locks are acquired, colored by thread. Bold the first edge per thread.

T1	T2
-----	-----
Lock(a);	Lock(c);
Lock(b);	Lock(a);
Lock(c);	Lock(b);
...	...



Notice the cycle in the graph!

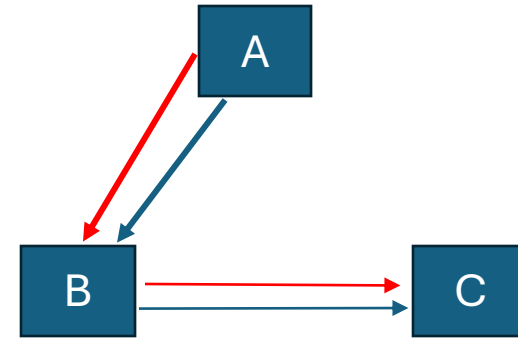
C -> A -> B -> C (note: the first edge of each color in the cycle is bolded.)

This gives the precise order of locks acquired that would cause a deadlock.

Deadlocks

Does this code cause a deadlock?

T1	T2
-----	-----
Lock(a);	Lock(a);
Lock(b);	Lock(b);
Lock(c);	Lock(c);
...	...

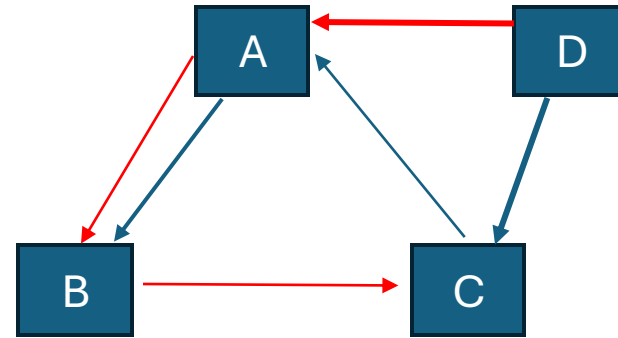


No, and we can see that the graph is cycle free.

Deadlocks

What about this?

T1	T2
-----	-----
Lock(d);	Lock(d);
Lock(a);	Lock(c);
Lock(b);	Lock(b);
Lock(c);	Lock(a);
...	...



No. There are cycles, but not any whose first red & blue edges are bold.

Deadlocks

And this?

T1

Lock(a);

Lock(a);

...



A thread can deadlock itself.

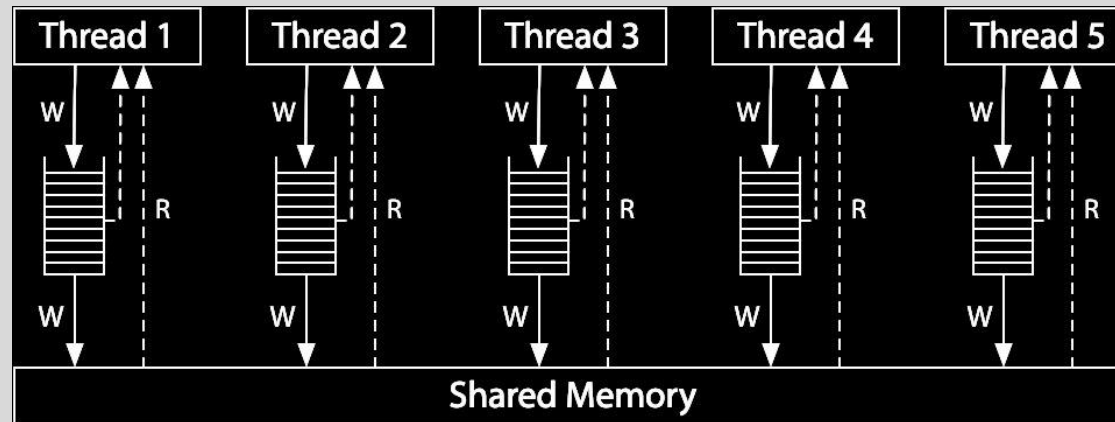
Deadlocks

Does the order of unlocking matter? Why or why not?

T1 ----- Lock(a); Lock(b); Lock(c); ... Unlock(c); Unlock(b); Unlock(a);	T2 ----- Lock(a); Lock(b); Lock(c); ... Unlock(b); Unlock(c); Unlock(a);
---	---

Total Store Order

- Precise definition:
 - Single threaded code acts as normal (loads cannot be reordered before stores by the hardware)
 - Stores become visible to other threads after some time
 - All threads agree on the total order of stores
- Remember store queues!



Total Store Order

- Precise definition:
 - Single threaded code acts as normal (loads cannot be reordered before stores by the hardware)
 - Stores become visible to other threads after some time
 - All threads agree on the total order of stores

T1

X = 1

R1 = Y

T2

Y = 1

R2 = X

Say that X and Y are both 0 initially.

Under sequential consistency, will see one of:

- R1 = 1, R2 = 0
- R1 = 0, R2 = 1
- R1 = 1, R2 = 1

Under TSO, may also see:

- R1 = 0, R2 = 0

Total Store Order

- Keep in mind compiler optimizations!
- By default, compilers assume code is single threaded
- Instructions may be added, removed, or reordered so long as they do not change observable single threaded behavior.

T1	T2	
-----	-----	
X = 1	R1 = X	
Y = 1	R2 = Y	Can T2 see R1 = 0, R2 = 0 assuming compiler optimizations?

- Solution: use atomics!
 - Atomics define their own memory model which compilers adhere to.
 - In C11, the default is sequential consistency

Scheduling

Scheduling Goals & Design Space

Goals (often competing with each other)

- Throughput: maximize useful work/time
- Latency / Response time: minimize time to first result - important for realtime/interactive applications
- Fairness: each process gets its share on the CPU -> No Starvation
- Resource utilization: CPU should never be idle while runnable processes exist
- Cost Minimization

Key design dimensions

- Pre-emptive vs. Cooperative scheduling: can the OS forcibly take the CPU?
- Granularity (quantum): 10 ms vs. 100 ms = lower latency but more overhead
- Constraints: real-time deadlines, starvation prevention, priority inversion
- Tradeoff spaces : Can't simultaneously improve for everything, always tradeoffs

Scheduling Costs & Context Switching Overview

Costs of scheduling

- Context switch overhead
- Cache pollution (new process evicts previous process's hot cache lines)
- Scheduler time complexity depending on the implementation
- Predictability : unpredictable timing violates RTOS guarantees

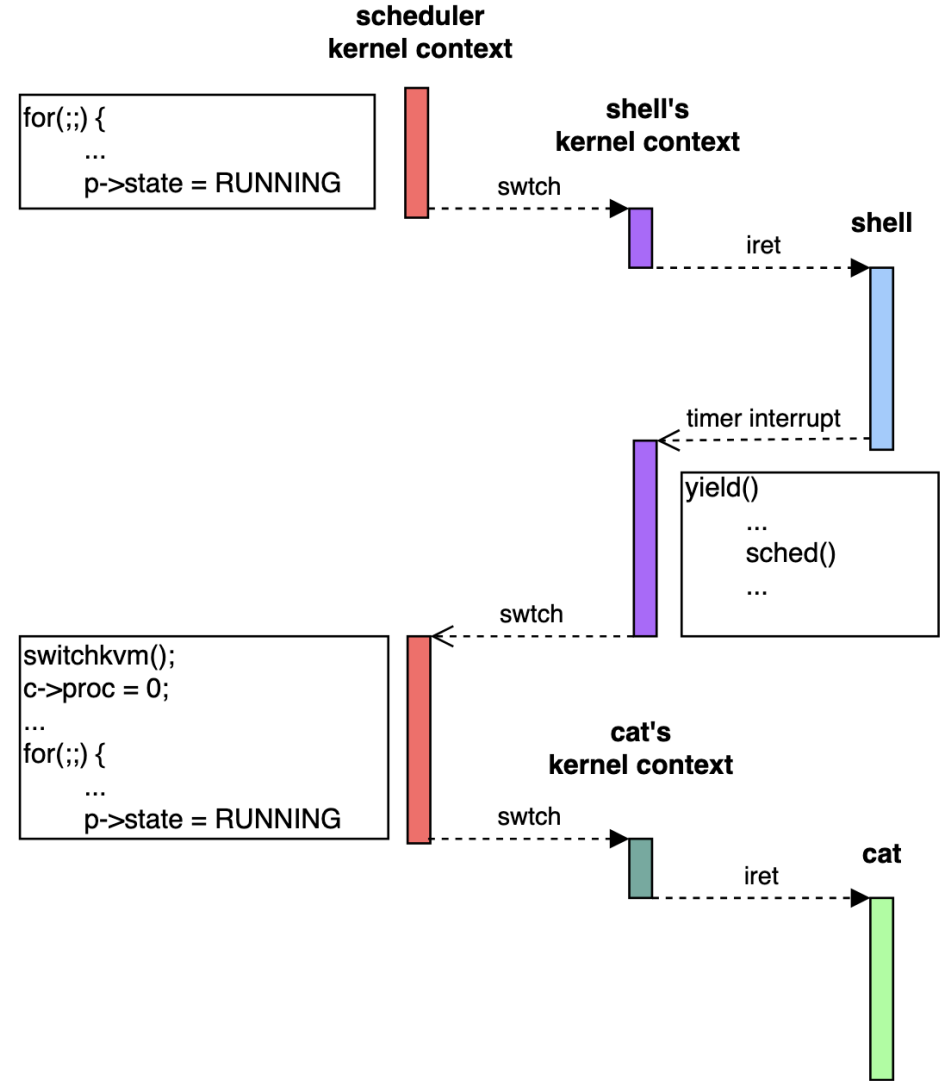
Benefits

- Better resource utilization : CPU never idles while runnable processes exist
- Priority-aware : higher-value work runs first
- Stronger QoS : latency SLOs, fairness, deadline support. Almost all workloads can be optimized with a proper workload aware scheduler.

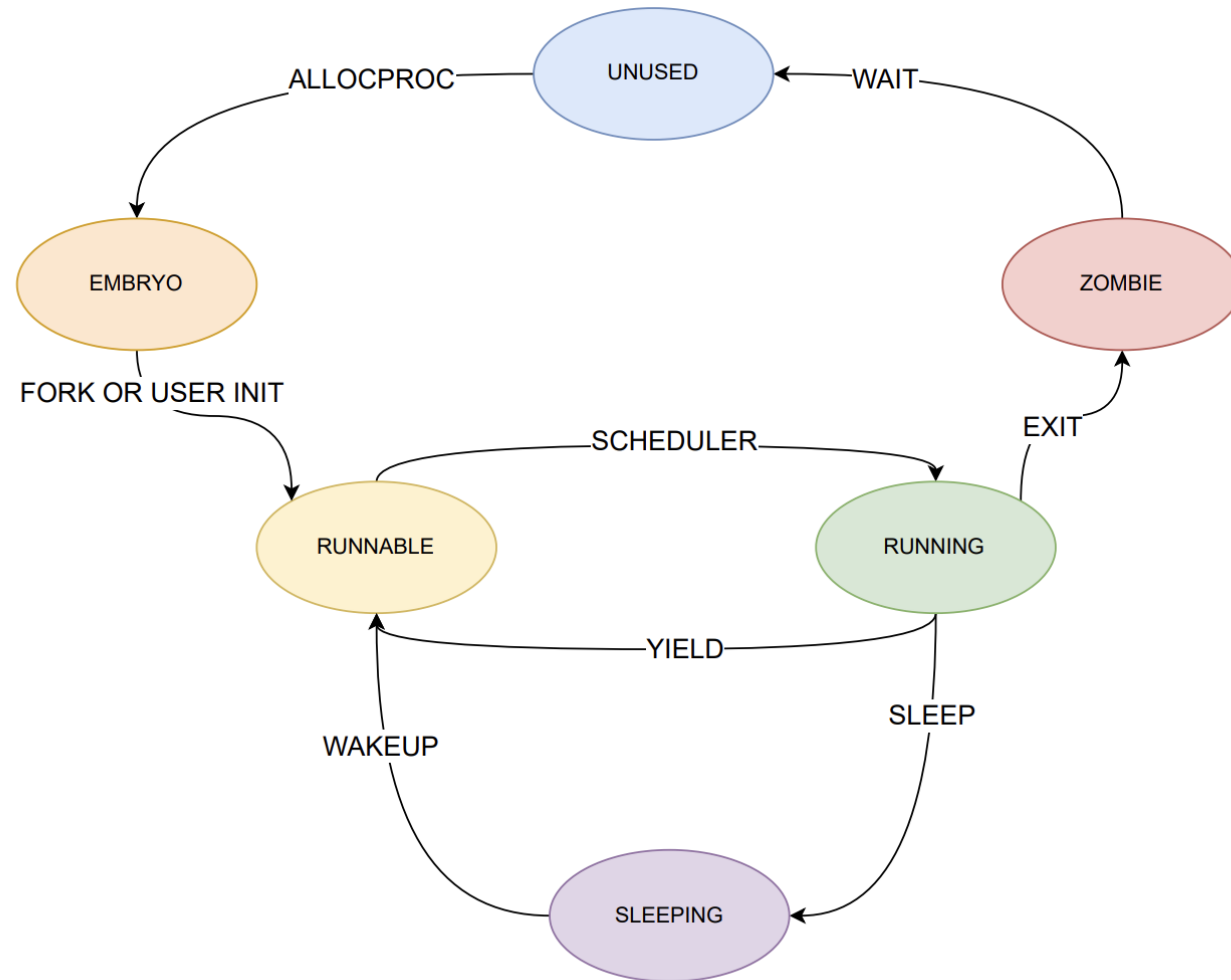
Context Switching — The Full Flow

*xv6 never switches directly from user-space to user-space.
Every context switch goes through the kernel.*

Full path: User Process A → trap/interrupt → yield() → sched()
→ swtch() → scheduler() → swtch() → new process kernel ctx
→ trap return → User Process B



xv6 process states and lifecycle



Scheduling Algorithms review

Algorithm	Pro	Con
FCFS (First Come First Served)	Simple; no starvation by arrival order	Convoy effect: short jobs stuck behind long ones; high avg. wait
SJF / SRTF (Shortest Job First)	Optimal avg. wait time; good for batch	Requires knowing job length; can starve long jobs
Round Robin (Fixed quantum per process)	Starvation-free; fair; simple	Doesn't distinguish urgency/priority; ignores deadlines
Priority (Run highest-priority first)	Urgency-aware; flexible via weights	Starvation of low-priority jobs; priority inversion possible
CFS — Linux default (Weighted Fair Queuing)	Proportional sharing; IO-bound processes get natural priority	More complex; vruntime accounting overhead

Linux CFS — Completely Fair Scheduler

Key Observation

Processes that constantly use full CPU are likely lower priority than those that mostly sleep
E.g., network servers need fast turnaround but spend most time waiting

Weighted Fair Queuing (WFQ) — Linux's CFS is an implementation

Resources shared proportionally to process weight

Example: Process A (weight 2) + Process B (weight 1) → A gets 2× the CPU

Virtual runtime (vruntime)

CFS tracks how much CPU time each process has used, normalized by weight

Always picks the process with the lowest vruntime to run next

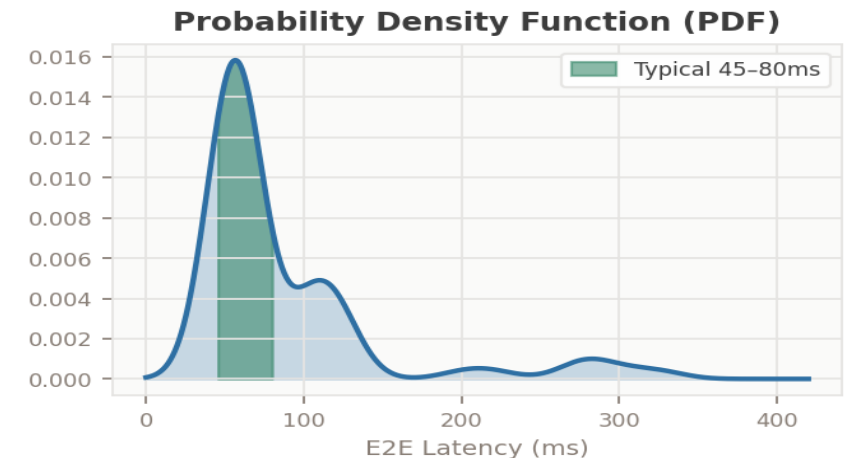
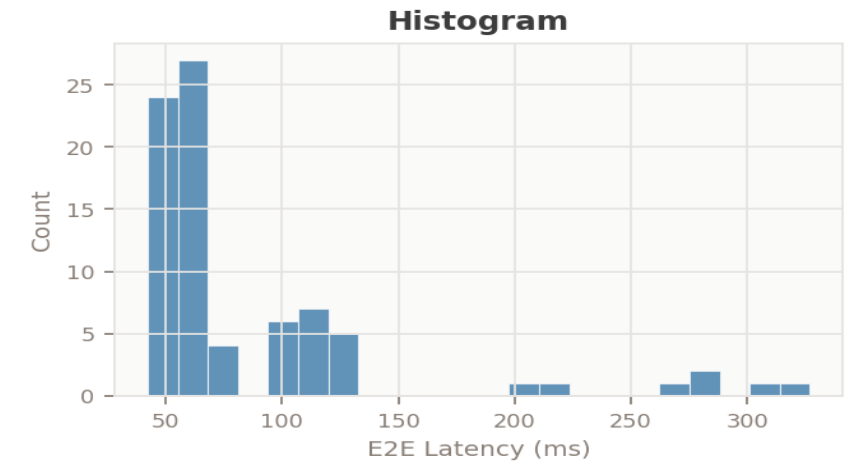
Sleeping processes

When a process sleeps, its vruntime freezes; on wake-up CFS resets it to min_vruntime

IO-bound processes naturally get priority — they never hogged the CPU while sleeping

Latency distribution

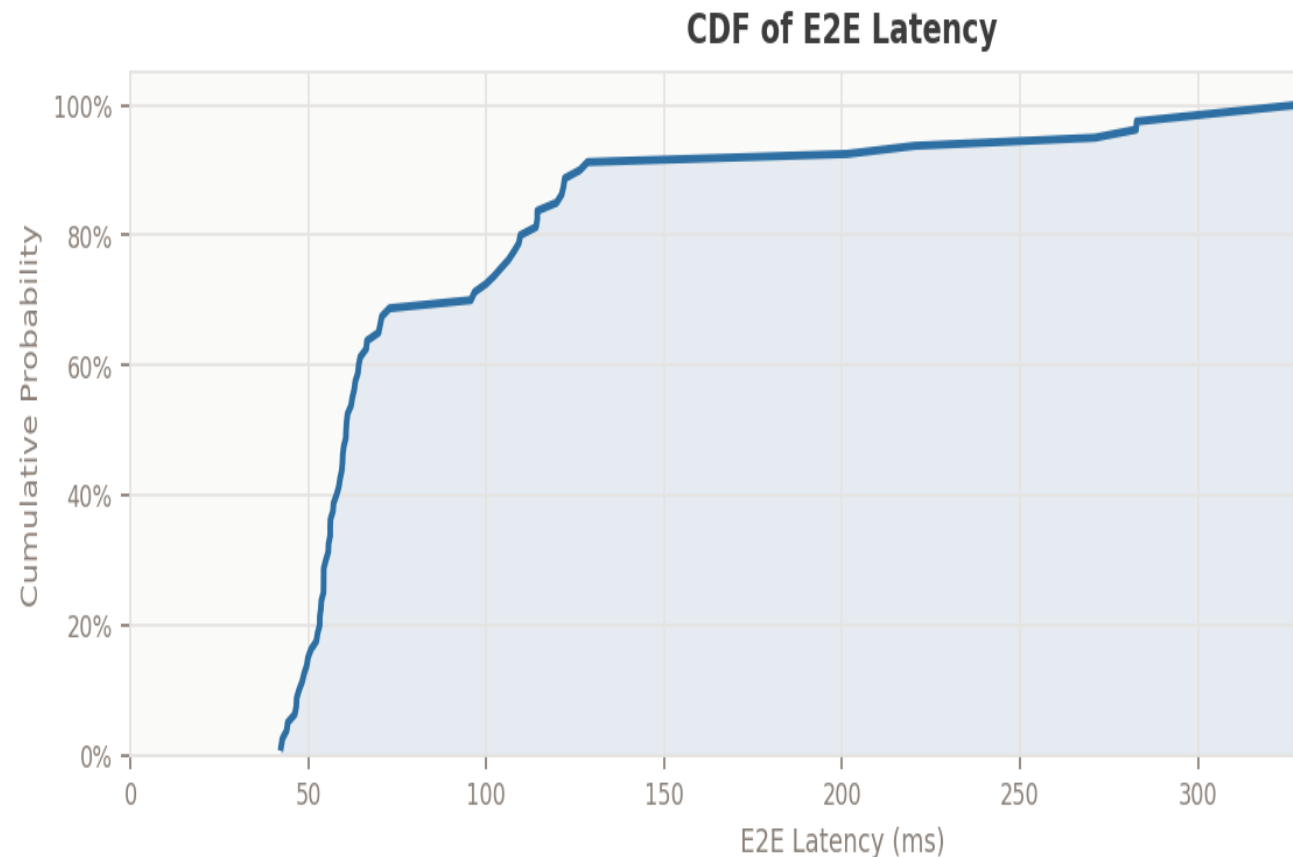
- Latency = $T_{\text{response}} - T_{\text{arrival}}$, usually measured in ms/ns across multiple queries or requests.
- Transforming from raw latency numbers to a Probability Distribution Function (PDF) generally requires bucketing into bins and then generating values or can be more continuous too.
 - $y = P(X=x)$
- We obtain the latency Cumulative Distribution Function (CDF) by integrating the PDF function above
 - $y = P(X \leq x)$
- Tail Latency: generally looking at the 99th ile values and comparing to a SLO Objective.



Latency distribution

Reading a CDF :

1. Pick a percentile value on the y-axis : So p90 translates to 0.9 on the y-axis on the CDF graph
2. Draw a horizontal line till it intersects with the CDF curve
3. Draw a vertical line down, this is the E2E latency at that particular percentile value
4. P90 of 32ms means that 90% of requests have an E2E latency of under 32ms.



Soft vs Hard Real-Time & Process characterization

Hard Real-Time: tasks must complete within a guaranteed deadline

Impractical with virtual memory or secondary storage - page faults stall for unbounded time

Soft Real-Time: critical tasks get priority over non-critical

Requires: (1) priority scheduling + (2) low dispatch latency (often NOT met - e.g., no preemption of syscalls)

Process characterization

- CPU-bound: consumes full quantum (compilation, ML training)
- IO-bound: frequently sleeps waiting on I/O (database, web server)
- Interactive: low CPU, latency-sensitive (shell, GUI) · Batch: background (cron, ETL)

Design Question

Question 1 : You're building a scheduler for a task with a sub 50ms deadline. Compare and reason about using different types of scheduling schemes like CFS/RR and EDF. Discuss the trade-offs for choosing each of these and what potential problems we might face ?

Question 2: Let's say now you have another lower priority task which is a long-running background task. You decided that FIFO with equal priority would be a good safe bet on this. Explain why this might or might not be a good idea ?

Design Question

Question 1 : You're building a scheduler for a task with a sub 50ms deadline. Compare and reason about using different types of scheduling schemes like CFS/RR and EDF. Discuss the trade-offs for choosing each of these and what potential problems we might face ?

- The issue with a CFS/RR schemes are that they are priority un-aware and are designed more for fairness ensuring there is no starvation between processes.
- Something like EDF which is deadline-aware scheduler would be ideal, can't guarantee deadline hits but provides best-effort scheduling of tasks based on the deadline

Question 2: Let's say now you have another lower priority task which is a long-running background task. You decided that FIFO with equal priority would be a good safe bet on this. Explain why this might or might not be a good idea ?

- FIFO with equal priorities is not a good idea here in this case.
- The long running task can enter the queue first, and due to equal priorities even if the task with the 50ms deadline comes in the FIFO queue, the background task will not be pre-empted and would block the task leading to missed deadlines.